Hitesh Rathore(HR) Notes

LINKEDIN LINK:
https://www.linkedin.com/in/hitesh-rathore-835279305/

How the browser parses HTML internally

The HTML parser is one of the most complex pieces of software in existence. Unlike JSON or XML parsers which crash on errors, the HTML parser has thousands of rules to gracefully handle every broken document ever written — by design.

Why a special parser was needed

In the early 90s, every browser wrote its own parser. Documents were broken constantly.
If a browser crashed on bad HTML, users switched to a competitor.
So parsers evolved to fix errors, not reject them. This is not a bug — it's intentional.

What Early Parsers Actually Did

Early HTML parsers were responsible for:

01 Reading HTML text › 02 Detecting tags › 03 Understanding structure › 04 Creating internal page representation › 05 Rendering webpage on screen

Internal Flow

Let e.g: `<h1>Hello</h1>`
`<p>World</p>`

Parser internally did something like:

01 Read character by character
↓
02 Identify tags
↓
03 Understand nesting
↓
04 Create internal tree
↓
05 Send data to renderer
↓
06 Display webpage

Browser reads HTML character by character

Browser receives HTML as plain text like this:
`<h1>Hello</h1>`

Internally this is actually just a sequence of characters:
`<, h, 1, >, H, e, l, l, o, <, /, h, 1, >`

The parser processes these characters one by one.

Why Character-by-Character?

Because browser receives webpage as a stream of bytes from network.

means: Data coming continuously in small pieces.

What Browser Actually Receives

- When we open a website: Browser does NOT instantly receive full webpage at once.
- Instead: Server sends data little by little through network.
- Like:

Chunk-1
Chunk-2
Chunk-3
Chunk-4
...
→ This continuous flow is called: Stream

Example

Suppose webpage HTML is: `<h1>Hello</h1>`

Server may send it like this:
Chunk-1 : `<h1>`
Chunk-2 : `Hel`
Chunk-3 : `lo</h1>`
↓
Browser starts parsing immediately.

It DOESN'T wait for entire file.

Q) Why Streaming Is Important ?

- Because websites can be HUGE.

If browser waited for full download: Page loading would feel very slow.

- So browser does:

Receive bytes
↓
Parse immediately
↓
Render immediately
→ This is called: Incremental Parsing

Internal Flow: Server → Browser

01 Server
↓
02 TCP Packets
↓
03 Network Bytes Arrive
↓
04 TCP Reassembles Correct Order
↓
05 Continuous Byte Stream
↓
06 Encoding Decoder
↓
07 HTML Parser

Important Internal Concept:

Browser first receives: RAW BYTES
NOT characters.
→
Then browser:
1. decodes bytes using encoding (UTF-8 etc.)
2. converts bytes → characters
3. parser reads characters
4. creates DOM

Internet fundamentally transfers: bytes

NOT: webpages, images, videos, text

Q) So how actually data travels on Internet ?

At the deepest level, data flowing on a network is basically:
- Electric signals / light pulses / radio waves
- carrying binary bits (0s and 1s)
- from one machine to another.

Medium	Actual Signal
→ Ethernet Cable	Tiny electrical changes travel.
→ Fiber Optic	Laser light pulses travel. → Example: Light ON → 1 Light OFF → 0
→ WiFi	Electromagnetic radio waves travel through air.

1. Tokenization

Q) What Is Tokenization in HTML Parsing?

Tokenization is the stage where the browser converts: raw characters into: meaningful parser units called TOKENS

First-Principles Understanding

Suppose browser receives: `<h1>Hello</h1>`
Initially browser only sees: <, h, 1, >, H, e, l, l, o, <, /, h, 1, >

How does browser know:
- where tag starts?
- where tag ends?
- what is text?
- what is an attribute?

This is exactly the problem tokenization solves.

Relate with Real-world:
Imagine this English sentence: "I love HTML"
Before understanding grammar, brain first splits sentence into: I | love | HTML
→ These meaningful pieces are like: Tokens

Similarly HTML tokenizer breaks webpage into understandable units.

Why Tokenization Exists

Without tokenization: Parser would need to process every individual character separately forever
That would be chaotic.

So browser first groups characters into:
- start tags
- end tags
- text
- comments
- attributes

HTML parsing internally:

Characters
↓
Tokenizer
↓
Tokens
↓
Tree Builder
↓
DOM Tree

e.g: `<h1 class="title">Hello</h1>`

Tokenizer produces: StartTagToken:
tagName = h1
attribute:
class = title

CharacterToken: Hello

EndTagToken: h1

https://github.com/WebKit/webkit/blob/main/Source/WebCore/html/parser/HTMLTokenizer.cpp

DOM tree creation process

The DOM Tree Creation Process is the stage where the browser converts: HTML tokens into: actual tree-like objects in memory

DOM:Document Object Model

DOM is: browser-created objects in memory

| HTML file: instruction document
| DOM: live object graph inside browser memory

Q) Why Browser Uses Tree Structure ?

Because webpages are hierarchical. Example:
```
<div>
  <p>Hello</p>
</div>
```
↓
Browser creates: Document
```
└── html
    └── body
        └── div
            └── p
                └── "Hello"
```

Example Internal Representation: `<div id="box"></div>`
↓
```
Node {
    type: ELEMENT_NODE
    tagName: "div"
    attributes: { id: "box" }
    children: []
    parent: body
}
```

Note: DOM Tree Is Live
DOM is NOT static. JavaScript can modify it anytime:

Browser dynamically updates:
- .DOM
- .layout
- .rendering

Important DOM creation is incremental.

Browser does NOT wait for full HTML file.

It: parses + builds DOM continuously while downloading

Q) If browsers receive everything as binary/raw bytes from servers, and the encoding format like UTF-8 is itself written INSIDE the HTML document using:
`<meta charset="UTF-8">`
then how does the browser even understand the very first few HTML lines before reaching this charset declaration?
How does the browser know whether those incoming bytes should be interpreted as ASCII, UTF-8, UTF-16, or something else?
And if the browser needs the encoding format first in order to read text correctly, then how can it even read the `<meta charset>` tag itself?
What actually happens internally from the moment raw binary data arrives from the network until the browser starts understanding characters, HTML tags, and eventually builds the webpage?

ans:

Step 1 → Browser First Receives RAW BYTES

When a server sends an HTML page, it does NOT send: <html> directly as readable text.

Instead, everything is sent as: RAW BYTES

Example internally:
00111100 01101000 01110100 01101101 ...

These bytes travel through:
- electrical signals
- WiFi radio waves
- fiber-optic light pulses

across the internet.

So the browser initially only receives: binary data (bytes)
NOT actual readable text.

Step 2 → Browser Must Determine Encoding

Before parsing HTML, browser must answer:

How should these bytes be converted into characters?

Because the same bytes can mean different characters in different encodings.

Example: A byte sequence may represent:

English text in ASCII
Unicode characters in UTF-8
completely different symbols in another encoding

So browser first performs: Encoding Detection

Step 3 → Browser Usually Knows Encoding BEFORE HTML Parsing

Most of the time, the browser already knows encoding from: HTTP Headers

Before HTML body arrives, server sends HTTP response headers like:
```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
```

This tells browser: Decode upcoming bytes using UTF-8

So browser already knows encoding BEFORE reading HTML content.

Step 4 → How Browser Receives HTTP Headers

Browser and server communicate using:
- TCP
- TLS (HTTPS)
- HTTP

Internally: Browser
TCP Connection
↓
HTTP Request Sent
↓
Server Responds

Server sends:
```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

<!DOCTYPE html>
<html>
...
```

Important structure:
```
HTTP Response
├── Headers
└── Body
```

Headers arrive FIRST.

Then browser sees: `\r\n\r\n` (blank line)

This tells browser: Headers ended.
HTML body starts now.

So browser processes headers BEFORE HTML parsing begins.

Step 5 → Browser Starts Decoding Bytes

Now browser knows: Use UTF-8

So it converts bytes into characters:

Bytes
↓
UTF-8 Decoder
↓
Characters

Example:

3C 68 74 6D 6C 3E

becomes: <html>

NOW browser can understand text.

Step 6 → Browser Reads <meta charset>

After decoding starts, browser eventually reaches: <meta charset="UTF-8">

This acts as:
- confirmation
- fallback
- correction mechanism

Step 7 → What If HTTP Header Does NOT Specify Charset?

Then browser uses fallback methods.

Method 1 → BOM (Byte Order Mark)

Some files start with special bytes.

Example UTF-8 BOM: EF BB BF

Browser recognizes this signature immediately.

Method 2 → Browser Default Guessing

If no charset exists:

Browser temporarily assumes default encoding.

Today usually: UTF-8

Older browsers sometimes guessed:
- ISO-8859-1
- Windows-1252
- Shift-JIS

depending on region/site.

Now `<meta charset="...">` will act as confirmation.

When browser finally finds the meta tag with charset,
it confirms (or corrects) the encoding it is using.
If different, browser re-decodes the already received bytes
using the correct encoding.

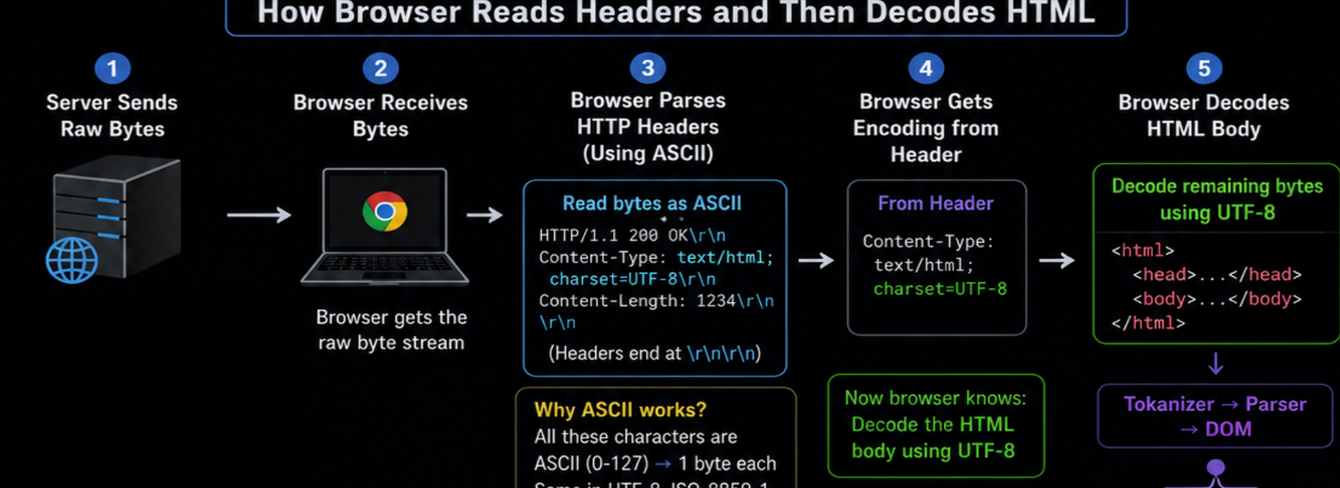
QUESTION: Browser gets encoding type in header then it convert .html file (byte code) in actual data then how it read that header?

ANSWER: Browser can read HTTP headers before knowing the encoding because HTTP headers are always encoded using ASCII-compatible characters.

→ HTTP is a text-based protocol that defines headers using only ASCII characters (0–127).
→ ASCII is a 7-bit encoding where each character is represented by 1 byte.
→ These 1-byte values are the same in UTF-8, ISO-8859-1, Windows-1252 and almost all encodings for characters 0–127.
→ So browser can safely read header bytes as ASCII without knowing the actual page encoding.
→ Once headers are read, browser looks at the Content-Type header (e.g., `charset=UTF-8`) to know how to decode the remaining HTML bytes.

Key Point:
Headers are parsed at the HTTP protocol layer, which is independent of the HTML document encoding.
Only after headers are parsed, the body (HTML) is decoded using the correct encoding.

How Browser Reads Headers and Then Decodes HTML


1 Server Sends Raw Bytes
2 Browser Receives Bytes
Browser gets the raw byte stream
3 Browser Parses HTTP Headers (Using ASCII)
Read bytes as ASCII
HTTP/1.1 200 OK\r\n
Content-Type: text/html;
charset=UTF-8\r\n
Content-Length: 1234\r\n
\r\n
(Headers end at \r\n\r\n)
Why ASCII works?
All these characters are ASCII (0-127) → 1 byte each
Same in UTF-8, ISO-8859-1, Windows-1252, etc.
4 Browser Gets Encoding from Header
From Header
Content-Type: text/html; charset=UTF-8
Now browser knows: Decode the HTML body using UTF-8
5 Browser Decodes HTML Body
Decode remaining bytes using UTF-8
<html>
<head>...</head>
<body>...</body>
</html>
Tokanizer → Parser → DOM
DOM Tree

In Short: 1. Headers use ASCII → browser can read without knowing encoding ▶ 2. Browser reads Content-Type header and finds charset ▶ 3. Browser decodes HTML body using that charset ▶ 4. Then parsing starts and DOM is built